



CSE 1112: Structured Programming Language Laboratory

Sidratul Tanzila Tasmi

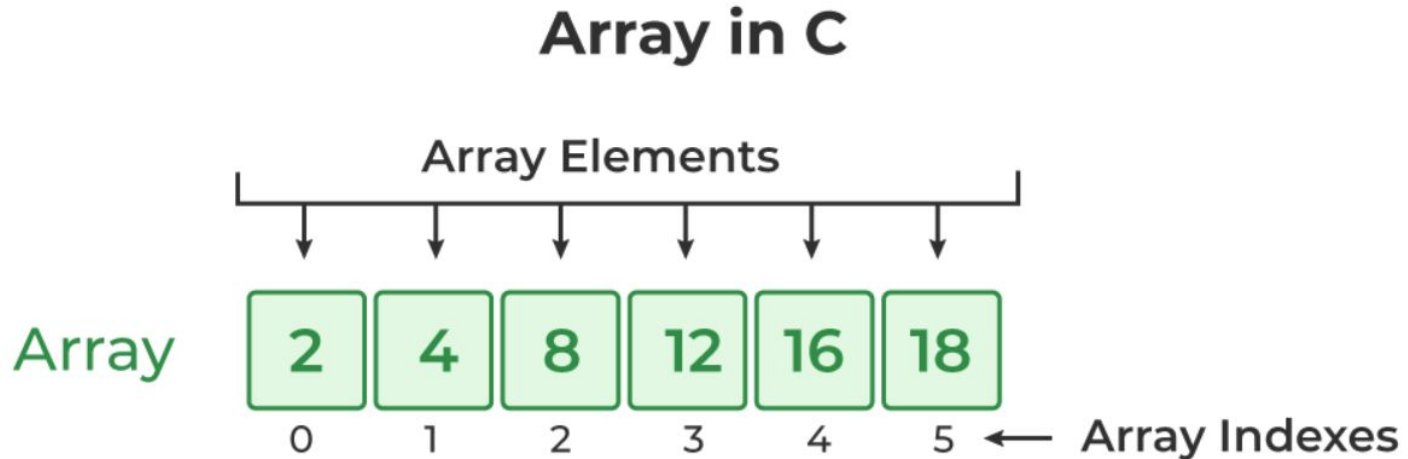
Lecturer, Department of Computer Science and Engineering
United International University

Array

Ref: <https://www.geeksforgeeks.org/c-arrays/>



- An array in C is a **fixed-size** collection of **similar data items** stored in **contiguous** memory locations.



Array Declaration

- Must be declared before using it.
- We can declare an array by specifying its **name**, **the type of its elements**, and **the size of its dimensions**.
- When we declare an array in C, the compiler allocates the memory block of the specified size to the array name.

Syntax of Array Declaration

```
data_type array_name [size];  
or  
data_type array_name [size1] [size2]...[sizeN];
```

Array Declaration

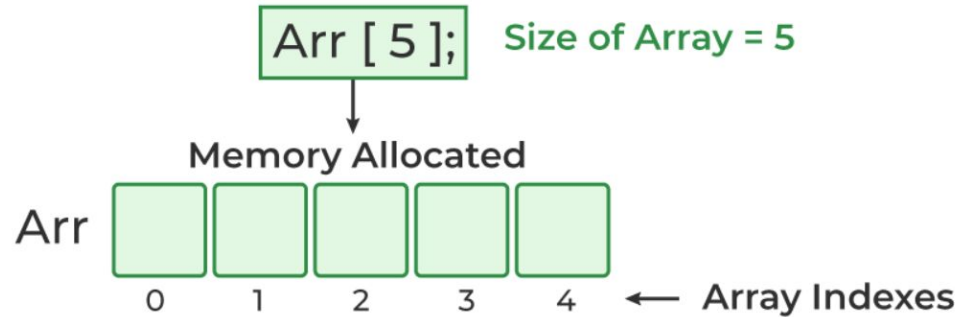


```
// C Program to illustrate the array declaration
#include <stdio.h>
```

```
int main()
{
    // declaring array of integers
    int arr_int[5];
    // declaring array of characters
    char arr_char[5];

    return 0;
}
```

Array Declaration



Array Initialization

- Initialization in C is the process to assign some initial value to the variable.
- When the array is declared or allocated memory, the elements of the array contain some garbage value. So, we need to initialize the array to some meaningful value.
- There are multiple ways in which we can initialize an array in C.



1. Array Initialization with Declaration



- In this method, we initialize the array along with its declaration.
- We use an initializer list to initialize multiple elements of the array. An initializer list is the list of values enclosed within braces { } separated by a comma.

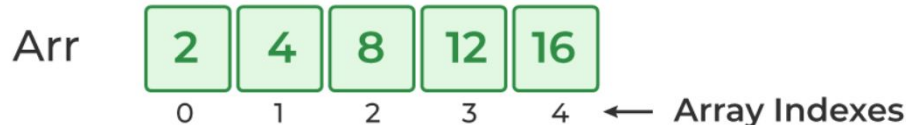
```
data_type array_name [size] = {value1, value2, ... valueN};
```

Array Initialization

Arr [5] = { 2, 4, 8, 12, 16 };



Memory Allocated and Initialized



2. Array Initialization with Declaration without Size



- If we initialize an array using an initializer list, we can skip declaring the size of the array as the compiler can automatically deduce the size of the array in these cases.
- The size of the array in these cases is equal to the number of elements present in the initializer list as the compiler can automatically deduce the size of the array.

data_type array_name[] = {value1,value2,value3,value4,value5};

The size of the above array is 5 which is automatically deduced by the compiler.

3. Array Initialization after Declaration (Using Loops)

- We initialize the array after the declaration by assigning the initial value to each element individually. We can use for loop, while loop, or do-while loop to assign the value to each element of the array.

```
for (int i = 0; i < N; i++) {  
    array_name[i] = valuei;  
}
```


Example



```
// C Program to demonstrate array initialization
#include <stdio.h>
int main()
{

    // array initialization using initializer list
    int arr[5] = { 10, 20, 30, 40, 50 };

    // array initialization using initializer list without
    // specifying size
    int arr1[] = { 1, 2, 3, 4, 5 };

    // array initialization using for loop
    float arr2[5];
    for (int i = 0; i < 5; i++) {
        arr2[i] = (float)i * 2.1;
    }
```

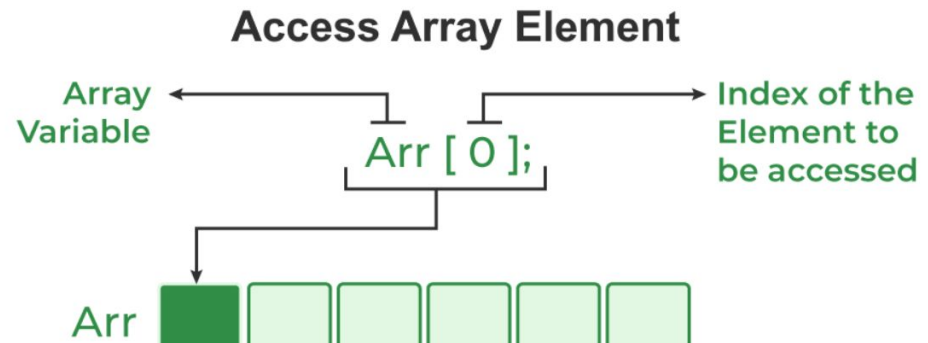
Access Array Elements



- We can access any element of an array in C using the array subscript operator `[]` and the index value `i` of the element.

array_name [index];

- One thing to note is that the **indexing in the array always starts with 0**, i.e., the first element is at index 0 and the last element is at $N - 1$ where N is the number of elements in the array.



Example



```
// C Program to illustrate element access using array subscript
#include <stdio.h>
int main()
{
    // array declaration and initialization
    int arr[5] = { 15, 25, 35, 45, 55 };

    // accessing element at index 2 i.e 3rd element
    printf("Element at arr[2]: %d\n", arr[2]);

    // accessing element at index 4 i.e last element
    printf("Element at arr[4]: %d\n", arr[4]);

    // accessing element at index 0 i.e first element
    printf("Element at arr[0]: %d", arr[0]);

    return 0;
}
```

Output

```
Element at arr[2]: 35
Element at arr[4]: 55
Element at arr[0]: 15
```

Update Array Element

- We can update the value of an element at the given index i in a similar way to accessing an element by using the array subscript operator `[]` and assignment operator `=`.

array_name[i] = new_value;



Array Traversal



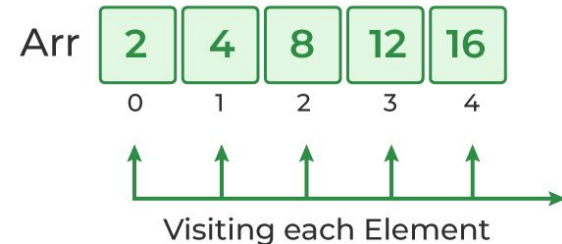
- Traversal is the process in which we visit every element of the data structure. For C array traversal, we use loops to iterate through each element of the array.

- Array Traversal using for Loop

```
for (int i = 0; i < N; i++) {  
    array_name[i];  
}
```

Array Transversal

```
for ( int i = 0; i < Size; i++){  
    arr[i];  
}
```



Example



```
// C Program to demonstrate the use of array
#include <stdio.h>
```

```
int main()
{
    // array declaration and initialization
    int arr[5] = { 10, 20, 30, 40, 50 };

    // modifying element at index 2
    arr[2] = 100;

    // traversing array using for loop
    printf("Elements in Array: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

```
Elements in Array: 10 20 100 40 50
```

Types of Array in C

There are two types of arrays based on the number of dimensions it has. They are as follows:



- One Dimensional Arrays (1D Array)
- Multidimensional Arrays

One Dimensional Array



- The One-dimensional arrays, also known as 1-D arrays in C are those arrays that have only one dimension.

1D Array

- Syntax of 1D Array:
array_name



Example



```
// C Program to illustrate the use of 1D array
#include <stdio.h>
```

```
int main()
{
```

```
    // 1d array declaration
    int arr[5];
```

```
    // 1d array initialization using for loop
    for (int i = 0; i < 5; i++) {
        arr[i] = i * i - 2 * i + 1;
    }
```

```
    printf("Elements of Array: ");
    // printing 1d array by traversing using for loop
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
```

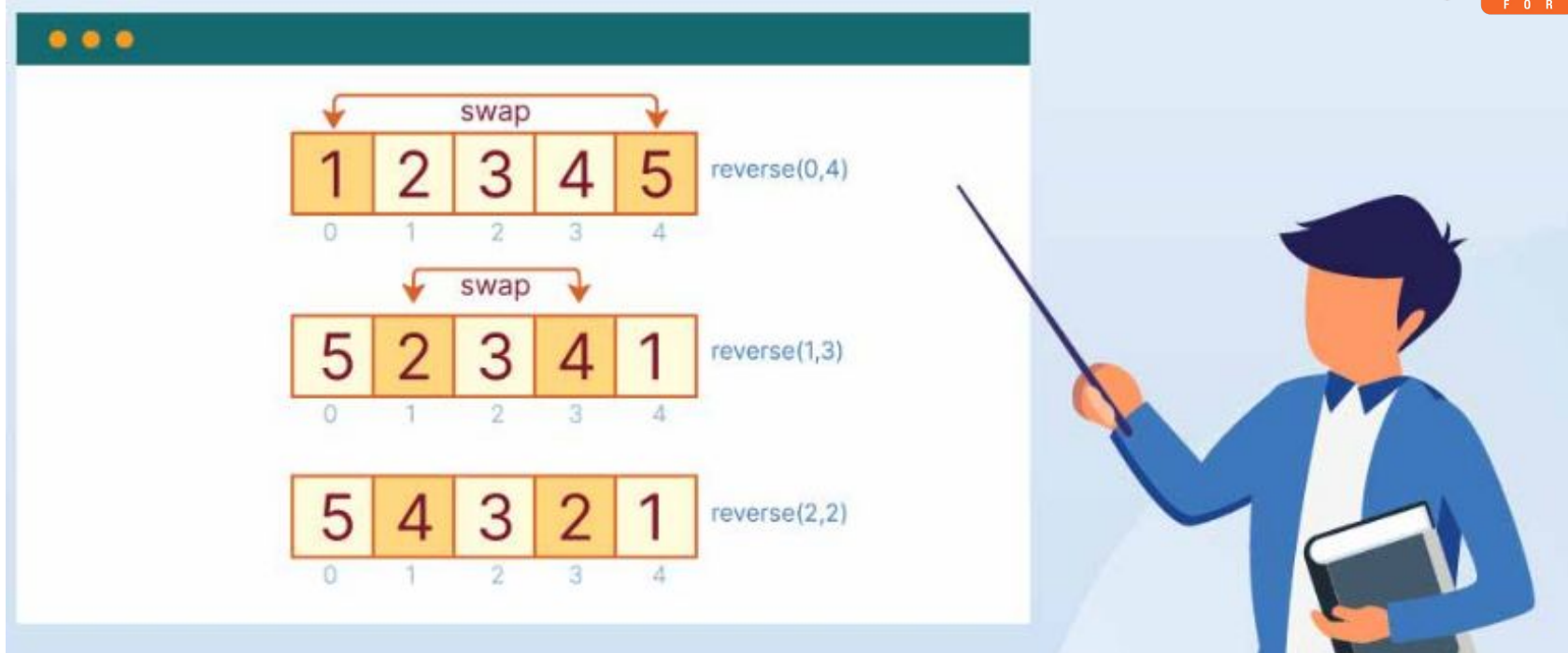
```
    return 0;
```

```
}
```

Output

Elements of Array: 1 0 1 4 9

Array reversal



Array reversal

```
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5}; // initialize the array
    int temp, i, j;
    printf("Original array: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    for (i = 0, j = 4; i < j; i++, j--) { // loop to swap elements
        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
    printf("\nReversed array: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
}
```

Output

```
/tmp/GIuMtzbQhI.o
Original array: 1 2 3 4 5
Reversed array: 5 4 3 2 1 |
```



Min max in array



Solution idea and steps

Step 1: We initialize two variables, **max** and **min**, with **X[0]** to store the maximum and minimum.

Step 2: Now we traverse the array from **i = 1** to **n - 1** and compare each element with **min** and **max**.

- **If ($X[i] < \text{min}$):** We have found a value **X[i]** smaller than the minimum so far. So, we update **min** with **X[i]**, i.e., **min = X[i]**.
- **If ($X[i] > \text{max}$):** We have found a value **X[i]** greater than the maximum so far. So, we update **max** with **X[i]**, i.e., **max = X[i]**.
- By the end of the loop, the minimum and maximum values of the array will be stored in the variables **min** and **max**.

Min max in array



```
#include <stdio.h>

int main() {
    int X[5] = {1, 2, 3, 4, 5}; // initialize the array
    int min, max;
    min = X[0];
    max = X[0];
    for(int i=1;i<5;i++){
        if(X[i]<min)
            min = X[i];
        else if(X[i]>max)
            max = X[i];
    }
    printf("Min = %d\nMax = %d",min,max);
    return 0;
```

Output

```
/tmp/GIuMtzbQhI.o
Min = 1
Max = 5
```



Thank you!